Assignment 8

Building Secure RESTful APIs with Node.js, Express, and MongoDB

Objective:

In this assignment, you will build RESTful APIs using **Node.js**, **Express**, and **MongoDB** to manage user information and handle image uploads. The assignment focuses on implementing endpoints for CRUD operations on users, secure password handling with **bcrypt**, user authentication, and enabling image uploads with file format validation. All testing will be performed through **Postman** without requiring a UI.

Assignment Goals:

- **Implement user-related API endpoints** using Node.js, Express, and MongoDB.
- **Securely store passwords** using bcrypt for hashing.
- **Enable image uploading** with validation for specific formats (e.g., JPEG, PNG, GIF) and uniqueness per user.
- **Implement user authentication** using email and password.
- **Test all endpoints** using Postman to verify functionality and correctness.
- **Swagger doc link:** https://app.swaggerhub.com/apis/northeasternuniversi-585/Users/1.0.0#/

Assignment Details:

1. User Creation

- **Endpoint**: POST /user/create
- **Functionality**:
  - Creates a new user with the following fields:
    - Full Name
    - Email
    - Password
  - **Validation**:
    - Ensure the email is in a valid format.
    - Validate the full name to allow only alphabetic characters.
    - Enforce a strong password rule (minimum 8 characters, including at least one uppercase letter, one lowercase letter, one digit, and one special character).
  - **Response**:
    - Success (201): {“message": "User created successfully." }
    - Error (400): {“error": "Validation failed." }

---

2. Update User Details

- **Endpoint**: PUT /user/edit
- **Functionality**:
  - Allows updating the following fields:
    - Full Name
    - Password
  - **Restrictions**:

        - Email cannot be updated.
    - **Validation**:
        - Validate the full name and password (same rules as above).
        - Ensure the user exists in the database before updating.
    - **Response**:
        - Success (200): {“message": "User updated successfully."}
        - Error (400): {"error": "Validation failed."}
        - Error (404): {"error": "User not found."}

---

3. Delete User

- **Endpoint**: DELETE /user/delete
- **Functionality**:
    - Deletes a user from the database by their email.
    - **Response**:
        - Success (200): {"message": "User deleted successfully."}
        - Error (404): {"error": "User not found."}

---

4. Retrieve All Users

- **Endpoint**: GET /user/getAll
- **Functionality**:
    - Retrieves a list of all users, including the following fields:
        - Full Name
        - Email Address
        - Password (hashed, as stored in the database)
    - **Response**:
        - Success (200):
        - {"users": [{"fullName": "John Doe", "email": "john.doe@example.com" }]}

---

5. Upload Image

- **Endpoint**: POST /user/uploadImage
- **Functionality**:
    - Allows a user to upload an image file specific to their account, identified by their email address.
    - **Validation**:
        - Accepts only one image file per user.
        - If a user tries to upload more than one image, return an error: "Image already exists."
        - Accepts only JPEG, PNG, and GIF file formats.
    - **Storage**:
        - Save the image in an "images" folder on the server.
        - Store the image file path along with the user's email in the database.
    - **Request Body**:
        - Form Data:
            - email (string, required): The email address of the user.
            - image (file, required): The image file to be uploaded.
    - **Response**:
        - Success (201): { "message": "Image uploaded successfully.", "filePath": "/images/filename.extension" }
        - Error (400): { "error": "Invalid file format. Only JPEG, PNG, and GIF are allowed." }
        - Error (400): { "error": "Image already exists for this user." }

- Error (404): { "error": "User not found." }

Additional Requirements:

1. **Validation**:
   - All API inputs must include appropriate validations.
   - Respond with clear, detailed error messages for validation failures.
2. **Secure Password Storage**:
   - Use **bcrypt** to hash and store passwords securely.
3. **HTTP Status Codes and Messages**:
   - Ensure all API responses include appropriate HTTP status codes and meaningful messages.
4. **Swagger Documentation**:
   - Use Swagger (or OpenAPI) to document all API endpoints, including request/response formats, validation rules, and examples.
5. **Postman Testing**:
   - Test all endpoints using Postman.
   - Provide a collection file (exported from Postman) with sample requests for all endpoints.

Submission Instructions:

1. **GitHub Repository**:
   - Push your code to a **private GitHub repository** using your Northeastern email account.
2. **Canvas Submission**:
   - Submit a ZIP file containing your project folder.
   - Include the GitHub repository URL in the Canvas remarks.
3. **Deadline**: Late submissions will result in a grade reduction.

Evaluation Criteria:

1. **Functionality**: Correct implementation of all endpoints and validations.
2. **Security**: Proper use of bcrypt for password hashing.
3. **Code Quality**: Clean, modular, and maintainable code.
4. **Testing**: Comprehensive testing of all endpoints using Postman.
5. **Documentation**: Clear and complete Swagger documentation.

Good Luck! If you have any questions, please reach out to your instructor or teaching assistant.